

Universidad de Oriente	
Ciencia de datos	Clase 05 - Ejemplo 1 - Análisis exploratorio de datos

Se dispone de un conjunto de datos que contiene información clínica de 768 pacientes, descrita mediante variables como el número de embarazos, el nivel de glucosa, la presión arterial, el grosor de la piel, la insulina, el índice de masa corporal, la función de antecedentes familiares de diabetes y la edad. La variable Outcome indica si el paciente presenta diabetes (1) o no (0).

El objetivo es realizar un análisis exploratorio y preparar adecuadamente los datos, identificando valores incongruentes, datos faltantes y posibles observaciones anómalas. Posteriormente, se deberá construir un modelo de regresión logística que permita estimar la probabilidad de que un paciente presente diabetes a partir de sus características clínicas.

```
In [10]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from collections import Counter
```

Análisis descriptivo de los datos

```
In [13]: diab_df= pd.read_csv('diabetes.csv')
```

```
In [15]: diab_df
```

Out[15]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigre
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	
...	...	...	...	...	...	...	
763	10	101	76	48	180	32.9	
764	2	122	70	27	0	36.8	
765	5	121	72	23	112	26.2	
766	1	126	60	0	0	30.1	
767	1	93	70	31	0	30.4	

768 rows × 9 columns



In [17]: `diab_df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies           768 non-null    int64
1   Glucose               768 non-null    int64
2   BloodPressure         768 non-null    int64
3   SkinThickness         768 non-null    int64
4   Insulin               768 non-null    int64
5   BMI                   768 non-null    float64
6   DiabetesPedigreeFunction 768 non-null    float64
7   Age                   768 non-null    int64
8   Outcome               768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

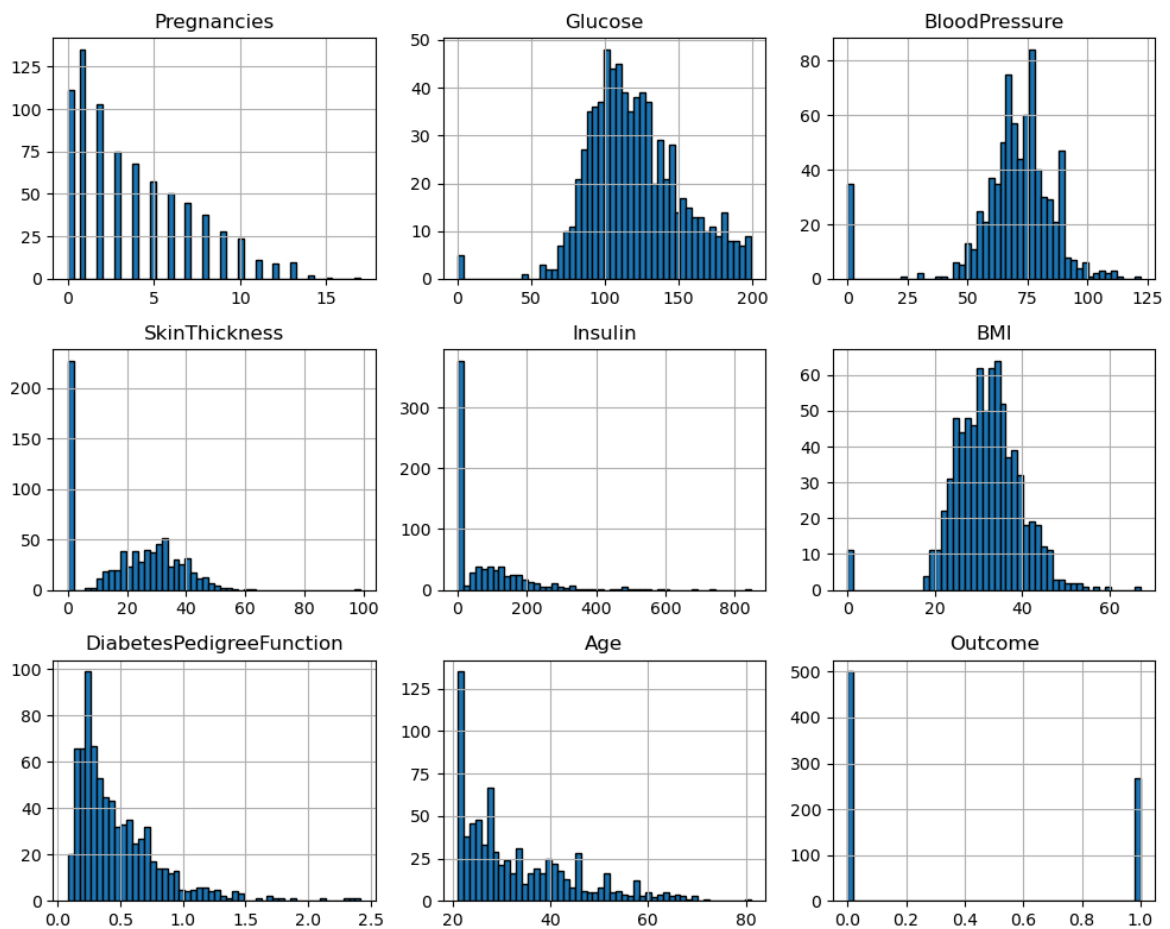
In [19]: `diab_df.isnull().sum()`

```
Out[19]: Pregnancies           0
Glucose             0
BloodPressure       0
SkinThickness       0
Insulin             0
BMI                 0
DiabetesPedigreeFunction 0
Age                 0
Outcome             0
dtype: int64
```

In [21]: `import matplotlib.pyplot as plt`

```
plt.figure()
diab_df.hist(figsize=(10,8),bins=50, edgecolor="black")
plt.tight_layout()
plt.show()
```

<Figure size 640x480 with 0 Axes>



Su concluye que no hay valores nulos en las variables. Sin embargo, los histogramas evidencian la presencia de valores incongruentes en las variables Glucose, BloodPressure, SkinThickness, Insulin y BIM. Por lo que esas incongruencias serán reemplazadas con valores nulos y luego se imputarán con la media.

```
In [23]: #Reemplazar valores incongruentes
diab_df[['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']] =diab_d
'SkinThi
```

```
In [25]: diab_df
```

Out[25]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigre
0	6	148.0	72.0	35.0	NaN	33.6	
1	1	85.0	66.0	29.0	NaN	26.6	
2	8	183.0	64.0	NaN	NaN	23.3	
3	1	89.0	66.0	23.0	94.0	28.1	
4	0	137.0	40.0	35.0	168.0	43.1	
...	...	...	...	...	...	...	
763	10	101.0	76.0	48.0	180.0	32.9	
764	2	122.0	70.0	27.0	NaN	36.8	
765	5	121.0	72.0	23.0	112.0	26.2	
766	1	126.0	60.0	NaN	NaN	30.1	
767	1	93.0	70.0	31.0	NaN	30.4	

768 rows × 9 columns



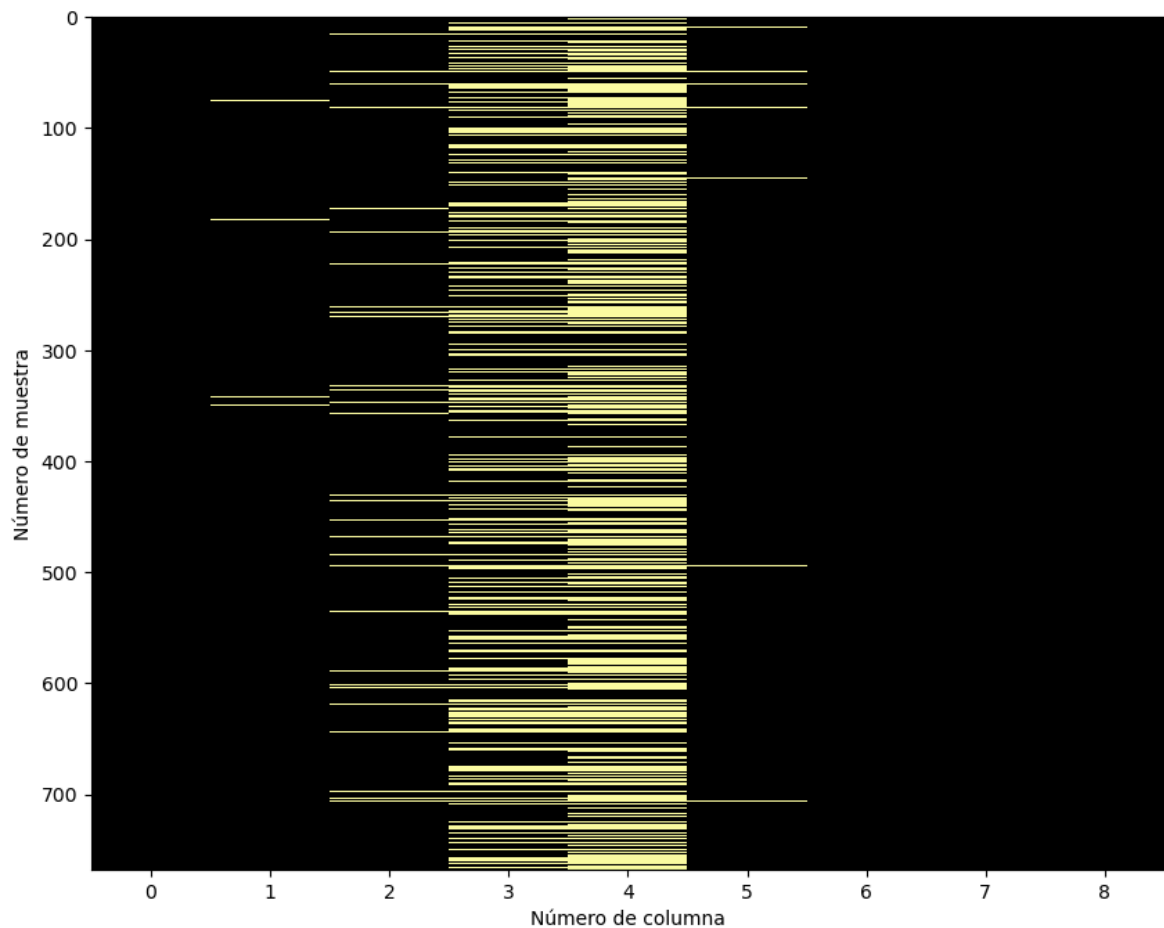
In [27]: `#Contabilizar los valores nulos`
`diab_df.isnull().sum()`

Out[27]:

Pregnancies	0
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

In [29]: `#Representar los valores nulos`
`plt.figure(figsize=(10, 8))`
`plt.imshow(diab_df.isna(), aspect="auto", interpolation="nearest", cmap="inferno")`
`plt.xlabel("Número de columna")`
`plt.ylabel("Número de muestra");`



```
In [31]: #Sustituir los valores nulos por la media de los datos

diab_df['Glucose'] = diab_df['Glucose'].fillna( diab_df['Glucose'].mean())
diab_df['BloodPressure'] = diab_df['BloodPressure'].fillna( diab_df['BloodPressure'].mean())
diab_df['SkinThickness'] = diab_df['SkinThickness'].fillna( diab_df['SkinThickness'].mean())
diab_df['Insulin'] = diab_df['Insulin'].fillna( diab_df['Insulin'].mean())
diab_df['BMI'] = diab_df['BMI'].fillna( diab_df['BMI'].mean())
```

```
In [33]: diab_df
```

Out[33]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPec
0	6	148.0	72.0	35.00000	155.548223	33.6	
1	1	85.0	66.0	29.00000	155.548223	26.6	
2	8	183.0	64.0	29.15342	155.548223	23.3	
3	1	89.0	66.0	23.00000	94.000000	28.1	
4	0	137.0	40.0	35.00000	168.000000	43.1	
...	...	...	...	...	...	...	
763	10	101.0	76.0	48.00000	180.000000	32.9	
764	2	122.0	70.0	27.00000	155.548223	36.8	
765	5	121.0	72.0	23.00000	112.000000	26.2	
766	1	126.0	60.0	29.15342	155.548223	30.1	
767	1	93.0	70.0	31.00000	155.548223	30.4	

768 rows × 9 columns

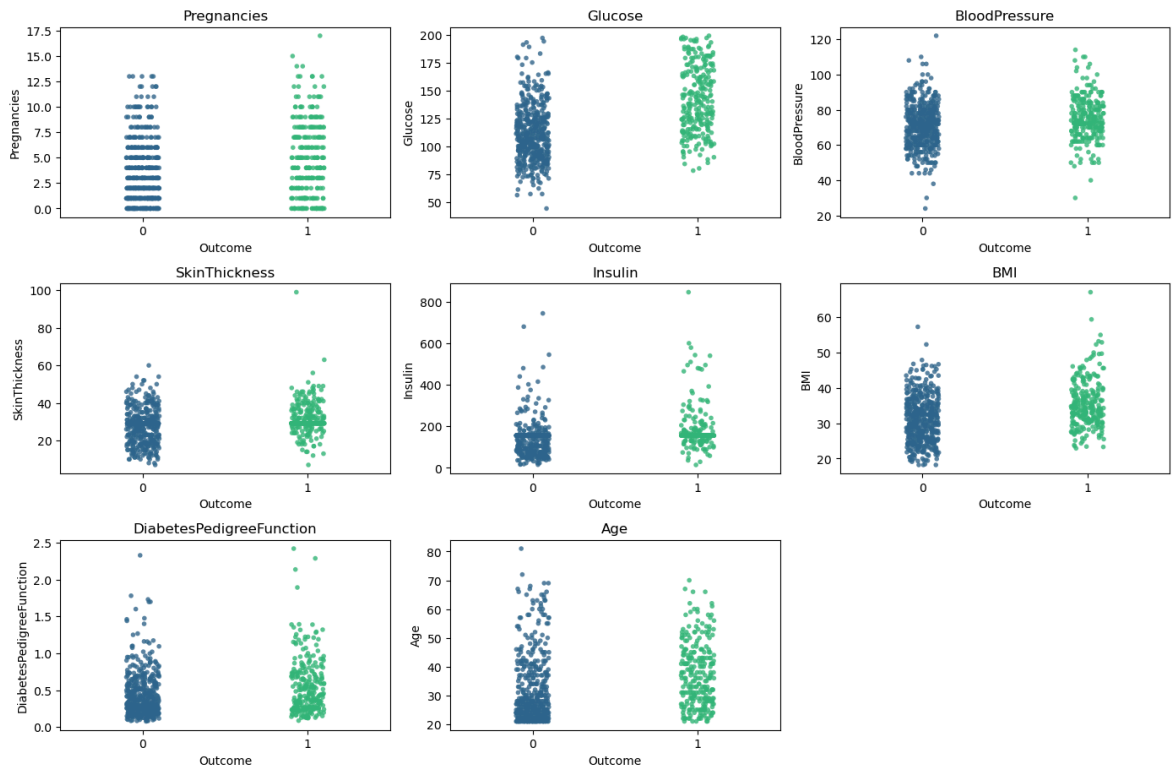


Diagramas de dispersión de enjambre

```
In [36]: columnas_numericas = diab_df.columns.tolist()
columnas_numericas.remove("Outcome")
n = len(columnas_numericas)

plt.figure(figsize=(14, 3*n))
for idx, var in enumerate(columnas_numericas, 1):
    plt.subplot(n, 3, idx)
    sns.stripplot(x="Outcome", y=var, hue="Outcome", data=diab_df, palette="viridi
jitter=True, alpha=0.8, size=4)
    plt.title(var)

plt.tight_layout()
plt.show()
```

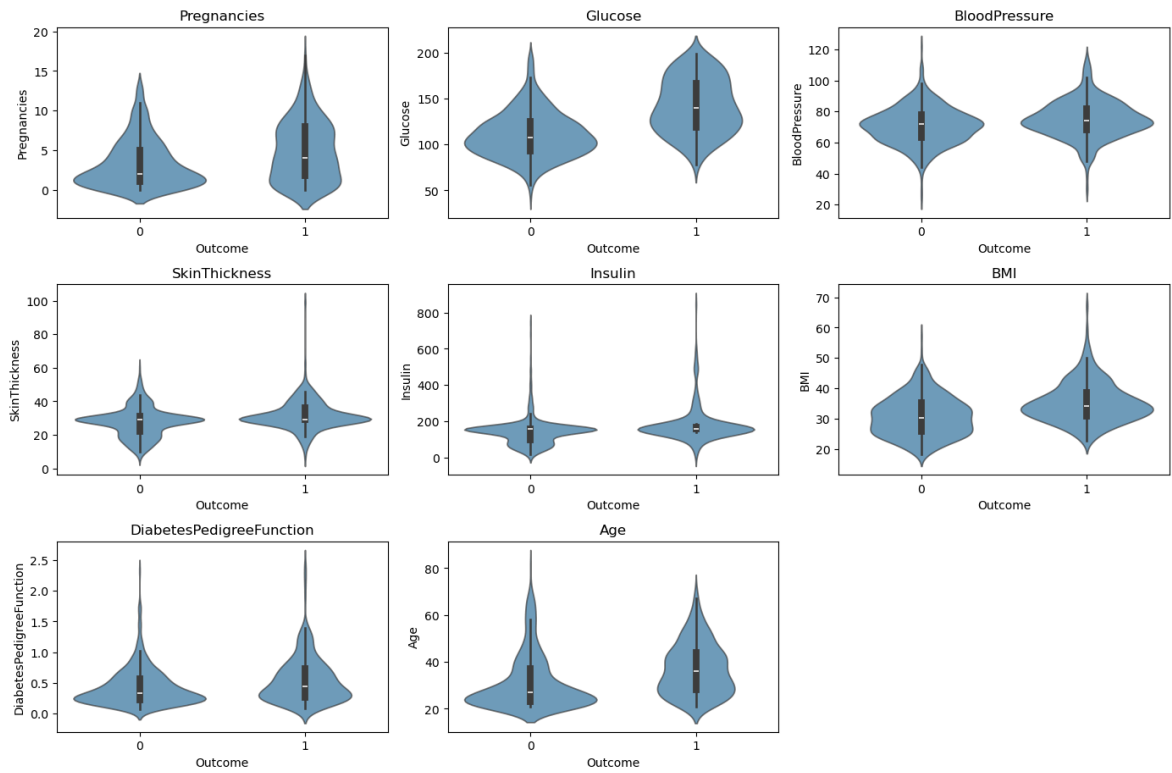


Diagramas de violín

```
In [38]: columnas_numericas = diab_df.columns.tolist()
columnas_numericas.remove("Outcome")
n = len(columnas_numericas)

plt.figure(figsize=(14, 3*n))
for idx, var in enumerate(columnas_numericas, 1):
    plt.subplot(n, 3, idx)
    sns.violinplot(x="Outcome", y=var, data=diab_df, alpha=0.7)
    plt.title(var)

plt.tight_layout()
plt.show()
```

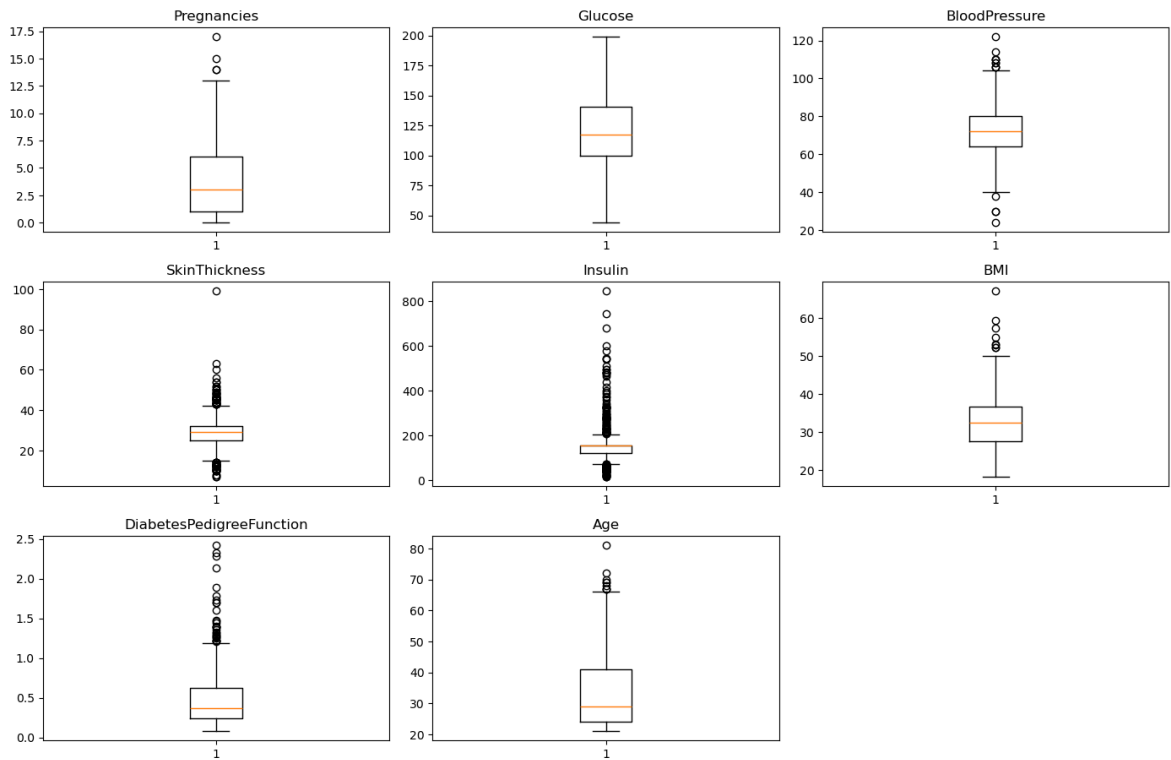


Análisis de los valores atípicos

```
In [40]: columnas_numericas = diab_df.columns.tolist()
columnas_numericas.remove("Outcome")
n = len(columnas_numericas)

plt.figure(figsize=(14, 3*n))
for idx, var in enumerate(columnas_numericas, 1):
    plt.subplot(n, 3, idx)
    plt.boxplot(diab_df[var])
    plt.title(var)

plt.tight_layout()
plt.show()
```

Los gráficos muestran que únicamente la variable Glucose no presenta valores atípicos, mientras que el resto de variables sí poseen.

Métodos de detección de valores atípicos

```
In [42]: def outliers_IQR(x):
    Q1 = x.quantile(0.25)
    Q3 = x.quantile(0.75)
    IQR = Q3 - Q1
    li = Q1 - 1.5 * IQR
    ls = Q3 + 1.5 * IQR
    num_outliers = sum((x < li) | (x > ls))
    return num_outliers

def outliers_desv(x):
    mu = x.mean()
    sigma = x.std()
    li = mu - 3 * sigma
    ls = mu + 3 * sigma
    num_outliers = sum((x < li) | (x > ls))
    return num_outliers

def outliers_z(df, n, features):
    outlier_list = []

    for column in features:
        data_media = df[column].mean()
        data_std = df[column].std()
        umbral = 3
        z_score = abs((df[column] - data_media) / data_std)
        outlier_list_column = df[z_score > umbral].index
        outlier_list.extend(outlier_list_column)

    outlier_list = Counter(outlier_list)
```

```

multiple_outliers = list( k for k, v in outlier_list.items() if v > n )
df1 = df[z_score > umbral]

return df1.shape[0]

from scipy.stats import median_abs_deviation

def z_scoremod_method(df,n,var):
    outlier_list = []

    for column in var:

        data_media = df[column].mean()
        data_std = df[column].std()
        umbral = 3
        MAD = median_abs_deviation

        mod_z_score = abs(0.6745*(df[column] - data_media)/MAD(df[column]))
        outlier_list_column = df[mod_z_score > umbral].index
        outlier_list.extend(outlier_list_column)

    outlier_list = Counter(outlier_list)
    multiple_outliers = list( k for k, v in outlier_list.items() if v > n )
    df1 = df[mod_z_score > umbral]

    return df1.shape[0]

```

```

In [44]: for vars in columnas_numericas:
        num_outliers = outliers_IQR(diab_df[vars])
        print("El número de outliers de la variable %s es %s" %(vars, num_outliers))

```

```

El número de outliers de la variable Pregnancies es 4
El número de outliers de la variable Glucose es 0
El número de outliers de la variable BloodPressure es 14
El número de outliers de la variable SkinThickness es 87
El número de outliers de la variable Insulin es 164
El número de outliers de la variable BMI es 8
El número de outliers de la variable DiabetesPedigreeFunction es 29
El número de outliers de la variable Age es 9

```

```

In [46]: lista_IQR = [outliers_IQR(diab_df[vars]) for vars in columnas_numericas]
        lista_desv = [outliers_desv(diab_df[vars]) for vars in columnas_numericas]
        lista_z = [outliers_z(diab_df,1,[vars]) for vars in columnas_numericas]
        lista_modz = [z_scoremod_method(diab_df,1,[vars]) for vars in columnas_numericas]

        total_outliers = pd.DataFrame({"Variables": columnas_numericas, "Método IQR": lista_IQR,
                                       "Método desviación": lista_desv, "Método puntuación z": lista_z,
                                       "Método puntuación z modificado": lista_modz})

```

```

In [48]: total_outliers

```

Out[48]:

	Variables	Método IQR	Método desviación	Método puntuación Z	Método puntuación z modificado
0	Pregnancies	4	4	4	14
1	Glucose	0	0	0	0
2	BloodPressure	14	8	8	10
3	SkinThickness	87	4	4	45
4	Insulin	164	19	19	344
5	BMI	8	5	5	5
6	DiabetesPedigreeFunction	29	11	11	28
7	Age	9	5	5	16

```

In [50]: fig, axes = plt.subplots(2, 4, figsize=(18, 12))
fig.suptitle(
    'Valores atípicos por categoría', fontsize=16)

sns.boxplot(ax=axes[0, 0], x=diab_df['Outcome'], y=diab_df['Pregnancies'],
            hue=diab_df['Outcome'])
axes[0, 0].set_title("Diabetes Outcome vs Pregnancies", fontsize=12)

sns.boxplot(ax=axes[0, 1], x=diab_df['Outcome'], y=diab_df['Glucose'],
            hue=diab_df['Outcome'], palette=('#FFFF00', '#800080'))
axes[0, 1].set_title("Diabetes Outcome vs Glucose", fontsize=12)

sns.boxplot(ax=axes[0, 2], x=diab_df['Outcome'], y=diab_df['BloodPressure'],
            hue=diab_df['Outcome'], palette=('#FFFF00', '#800080'))
axes[0, 2].set_title("Diabetes Outcome vs BloodPressure", fontsize=12)

sns.boxplot(ax=axes[0, 3], x=diab_df['Outcome'], y=diab_df['SkinThickness'],
            hue=diab_df['Outcome'], palette=('#FFFF00', '#800080'))
axes[0, 3].set_title("Diabetes Outcome vs SkinThickness", fontsize=12)

sns.boxplot(ax=axes[1, 0], x=diab_df['Outcome'], y=diab_df['Insulin'],
            hue=diab_df['Outcome'], palette=('#FFFF00', '#800080'))
axes[1, 0].set_title("Diabetes Outcome vs Insulin", fontsize=12)

sns.boxplot(ax=axes[1, 1], x=diab_df['Outcome'], y=diab_df['BMI'],
            hue=diab_df['Outcome'], palette=('#FFFF00', '#800080'))
axes[1, 1].set_title("Diabetes Outcome vs BMI", fontsize=12)

sns.boxplot(ax=axes[1, 2], x=diab_df['Outcome'], y=diab_df['DiabetesPedigreeFunc
            hue=diab_df['Outcome'], palette=('#FFFF00', '#800080'))
axes[1, 2].set_title(
    "Diabetes Outcome vs DiabetesPedigreeFunction", fontsize=12)

sns.boxplot(ax=axes[1, 3], x=diab_df['Outcome'], y=diab_df['Age'],
            hue=diab_df['Outcome'], palette=('#FFFF00', '#800080'))
axes[1, 3].set_title("Diabetes Outcome vs Age", fontsize=12)

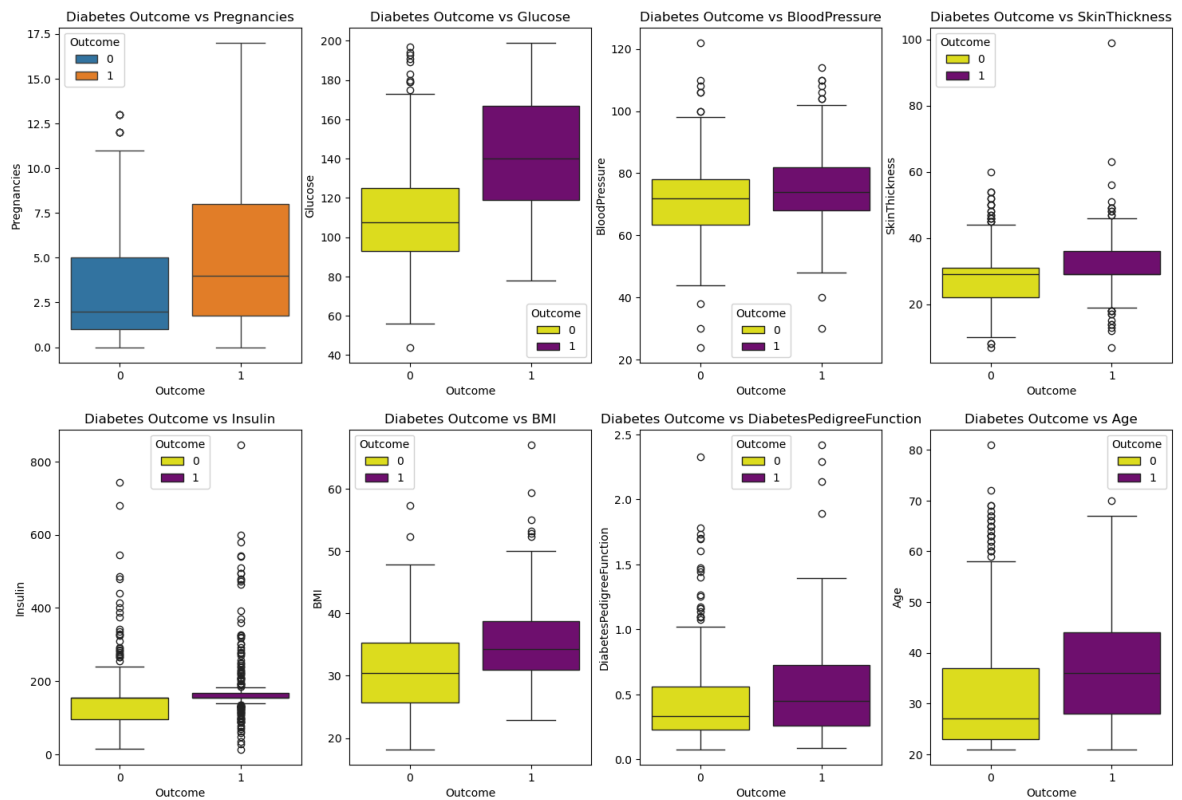
```

```

Out[50]: Text(0.5, 1.0, 'Diabetes Outcome vs Age')

```

Valores atípicos por categoría



Detección de anomalías

```
In [53]: from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.05, random_state=2026)

anomalias = iso.fit_predict(diab_df[columnas_numericas])
puntaje_anomalias = iso.decision_function(diab_df[columnas_numericas])

diab_df["Anomalías"] = anomalias
diab_df['Puntaje de anomalía'] = puntaje_anomalias
```

```
In [54]: diab_df
```

Out[54]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPec
0	6	148.0	72.0	35.00000	155.548223	33.6	
1	1	85.0	66.0	29.00000	155.548223	26.6	
2	8	183.0	64.0	29.15342	155.548223	23.3	
3	1	89.0	66.0	23.00000	94.000000	28.1	
4	0	137.0	40.0	35.00000	168.000000	43.1	
...	...	...	...	...	...	...	
763	10	101.0	76.0	48.00000	180.000000	32.9	
764	2	122.0	70.0	27.00000	155.548223	36.8	
765	5	121.0	72.0	23.00000	112.000000	26.2	
766	1	126.0	60.0	29.15342	155.548223	30.1	
767	1	93.0	70.0	31.00000	155.548223	30.4	

768 rows × 11 columns

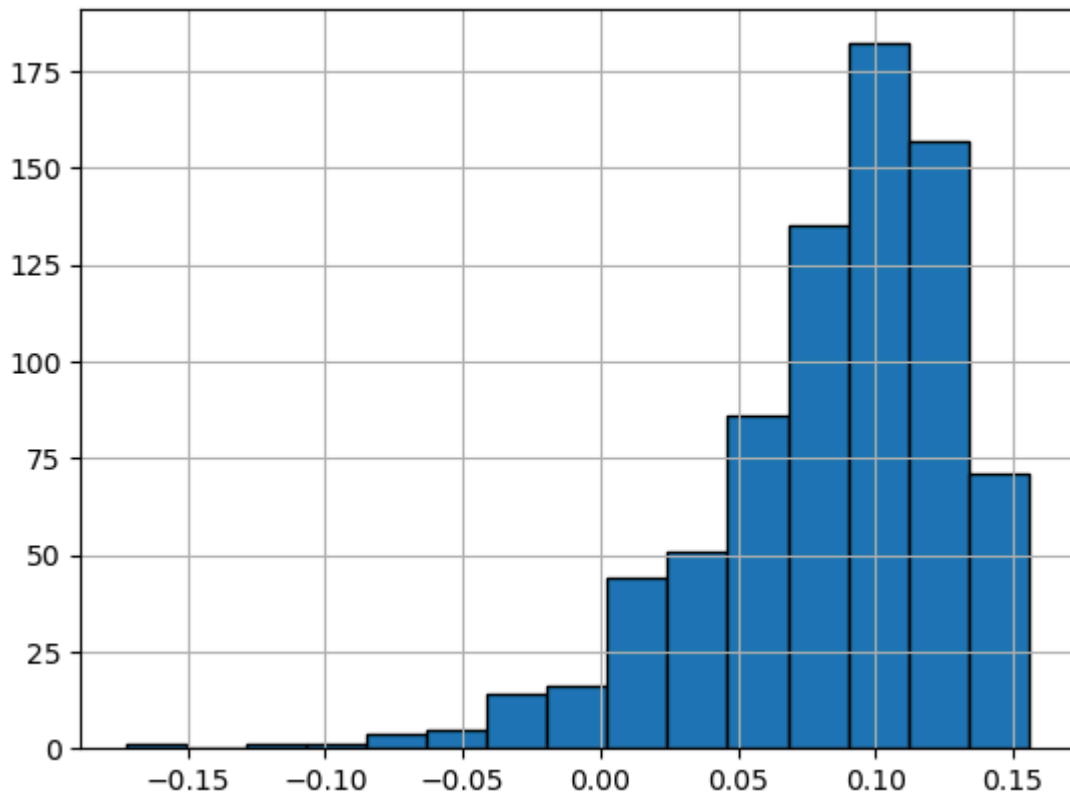
In [57]: `diab_df['Anomalías'].value_counts()`

Out[57]: Anomalías
 1 729
 -1 39
 Name: count, dtype: int64

El algoritmo Isolation Forest clasificó 729 observaciones, equivalentes al 94.92 % del conjunto de datos, como registros normales, mientras que identificó 39 observaciones, correspondientes al 5.08 %, como posibles anomalía

Distribución de las anomalías

```
In [61]: plt.figure()
diab_df['Puntaje de anomalía'].hist(bins=15, edgecolor="black")
plt.show()
```

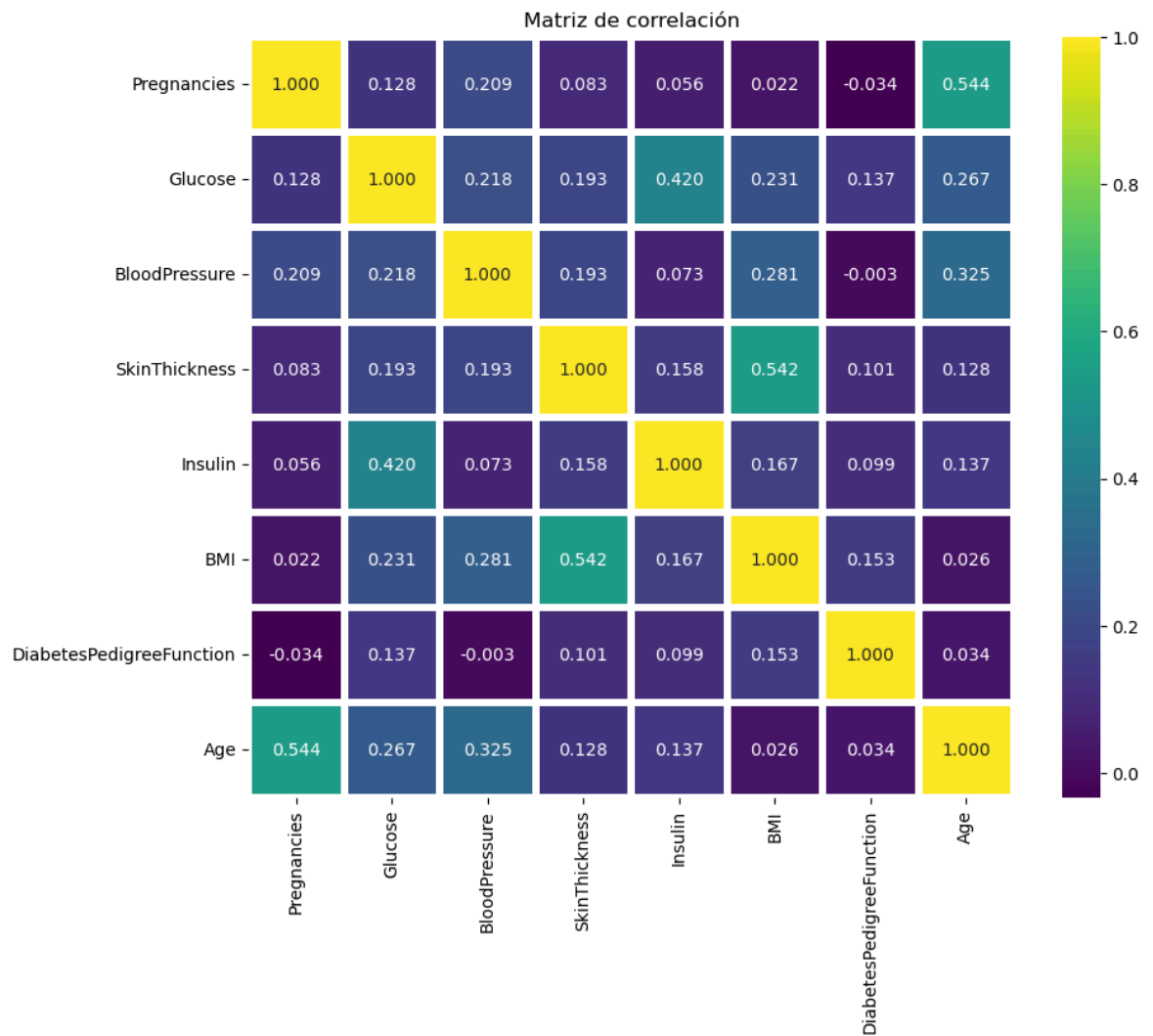


Matrices de correlación

```
In [63]: import seaborn as sns

plt.figure(figsize=(10, 8))
sns.heatmap(diab_df[columnas_numericas].corr(), annot=True, fmt='.3f', linewidth

plt.title('Matriz de correlación')
plt.show()
```

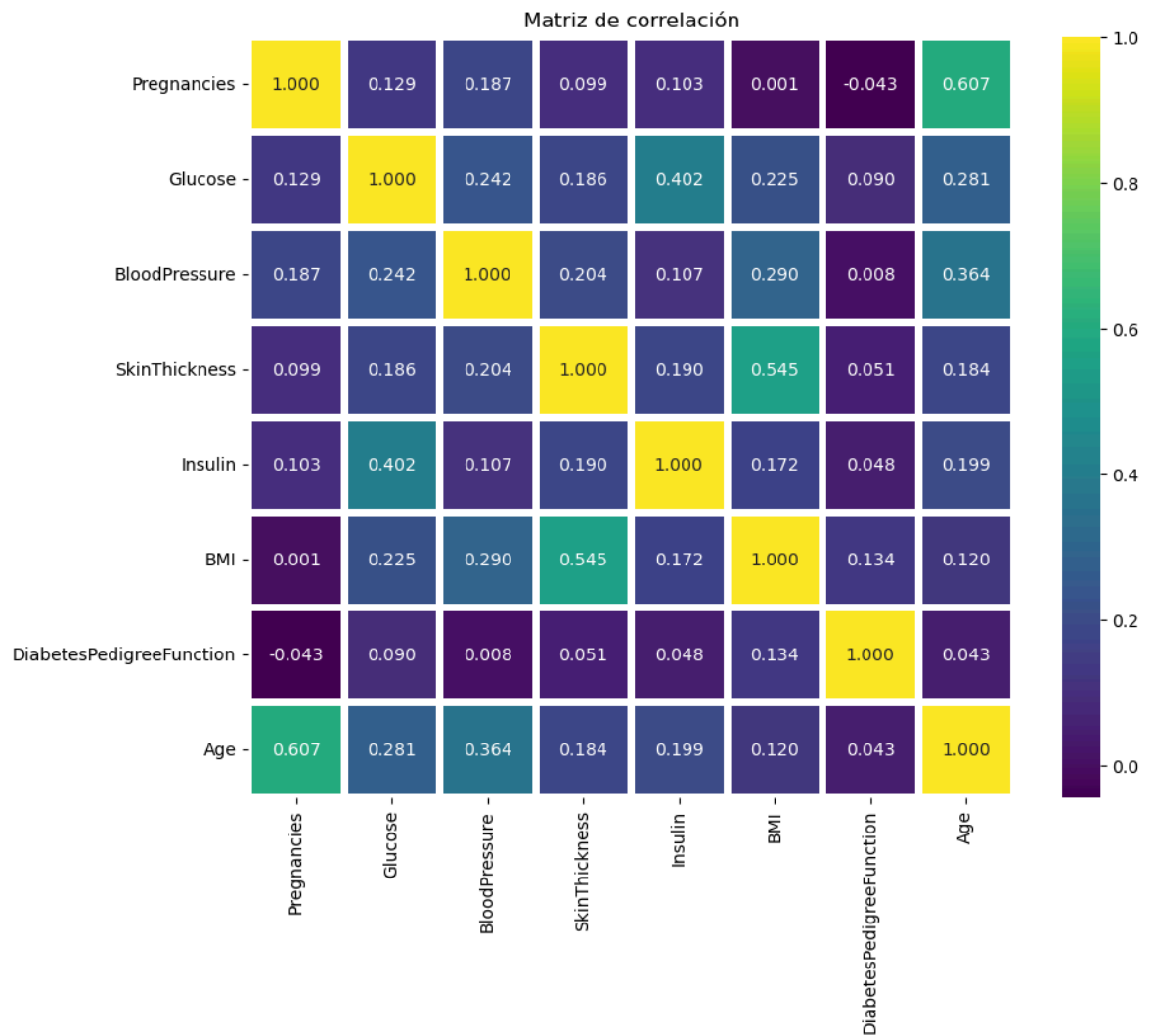


```
In [64]: import seaborn as sns

plt.figure(figsize=(10, 8))

sns.heatmap(diab_df[columnas_numericas].corr(method="spearman"), annot=True, fmt

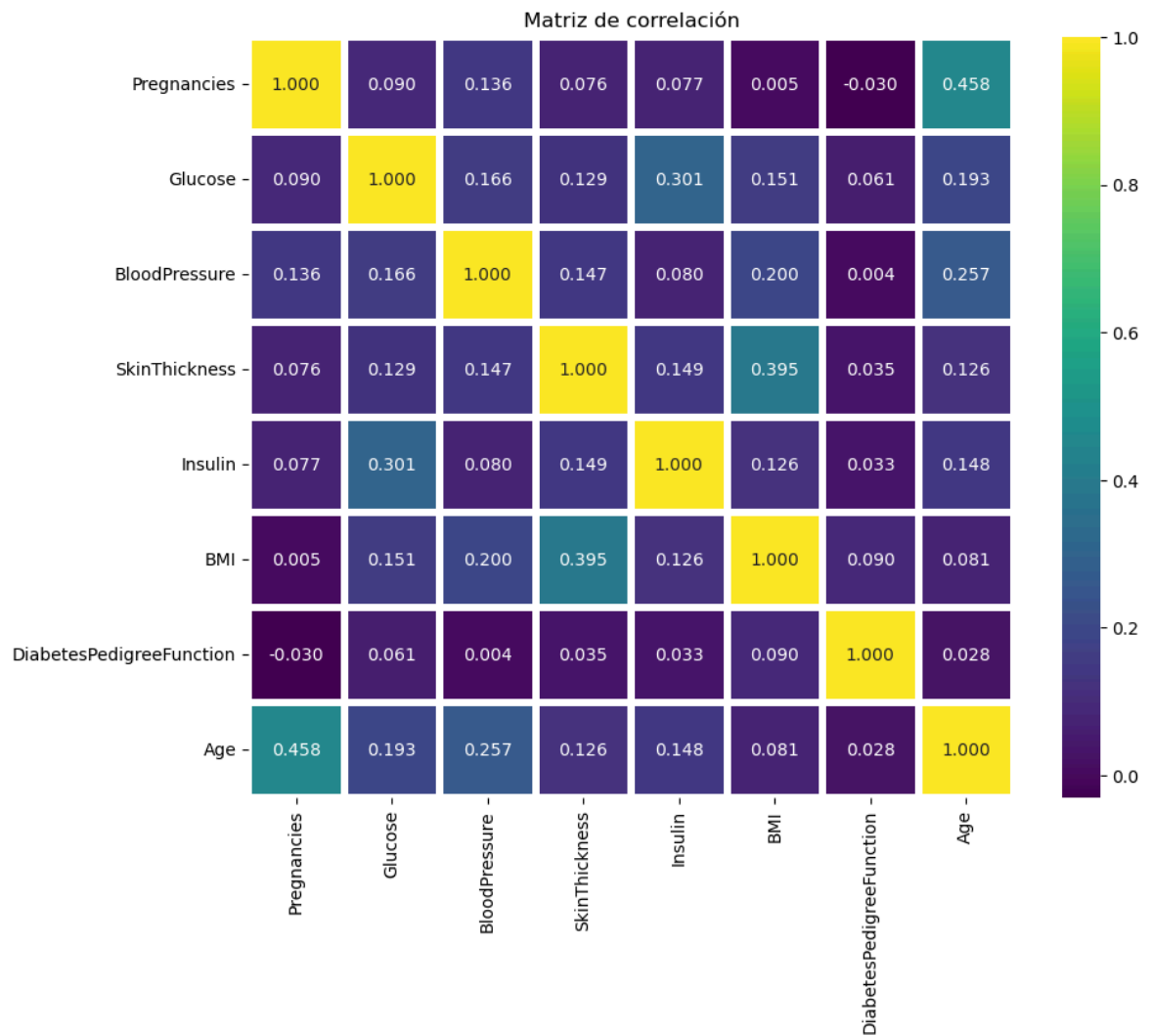
plt.title('Matriz de correlación')
plt.show()
```



```
In [66]: plt.figure(figsize=(10, 8))

sns.heatmap(diab_df[columnas_numericas].corr(method="kendall"), annot=True, fmt=

plt.title('Matriz de correlación')
plt.show()
```

Implementación de la regresión logística

```
In [70]: X_independientes = diab_df[columnas_numericas]
         y_dependiente = diab_df["Outcome"]
```

```
In [72]: from sklearn.linear_model import LogisticRegression

         logistic_reg = LogisticRegression(penalty=None, max_iter=150)
         logistic_reg.fit(X_independientes, y_dependiente)
```

```
Out[72]: LogisticRegression
         LogisticRegression(max_iter=150, penalty=None)
```

```
In [74]: #Mostrar el intercepto
         print("El intercepto de la regresión es %s" %logistic_reg.intercept_)
```

El intercepto de la regresión es [-9.09668033]

```
In [76]: #Mostrar los coeficientes
         coefs = pd.DataFrame({"Variables": columnas_numericas, "Coeficientes": logistic_reg.coef_})
```

Out[76]:

	Variables	Coeficientes
0	Pregnancies	0.125010
1	Glucose	0.037350
2	BloodPressure	-0.008804
3	SkinThickness	0.003483
4	Insulin	-0.000788
5	BMI	0.093110
6	DiabetesPedigreeFunction	0.866060
7	Age	0.013141

```
In [78]: import statsmodels.api as sm

#Añadir el intercepto
X2 = sm.add_constant(X_independientes)

#Ajustar el modelo
est = sm.Logit(y_dependiente, X2)

#ver el ajuste
est2 = est.fit()
print(est2.summary())
```

Optimization terminated successfully.

Current function value: 0.464388

Iterations 6

Logit Regression Results

```
=====
Dep. Variable:          Outcome    No. Observations:          768
Model:                  Logit      Df Residuals:              759
Method:                  MLE       Df Model:                  8
Date:                   Fri, 31 Jul 2026    Pseudo R-squ.:          0.2820
Time:                   10:08:09    Log-Likelihood:         -356.65
converged:              True        LL-Null:                 -496.74
Covariance Type:        nonrobust    LLR p-value:            6.750e-56
=====
```

```
=====
                                coef    std err          z      P>|z|      [0.025
0.975]
-----
const                -9.0968      0.813    -11.195     0.000    -10.689
-7.504
Pregnancies           0.1250      0.032     3.860     0.000     0.062
0.188
Glucose               0.0374      0.004     9.630     0.000     0.030
0.045
BloodPressure        -0.0088      0.009    -1.028     0.304    -0.026
0.008
SkinThickness         0.0035      0.013     0.265     0.791    -0.022
0.029
Insulin              -0.0008      0.001    -0.671     0.502    -0.003
0.002
BMI                   0.0931      0.018     5.219     0.000     0.058
0.128
DiabetesPedigreeFunction  0.8661      0.296     2.923     0.003     0.285
1.447
Age                   0.0131      0.010     1.382     0.167    -0.005
0.032
=====
```

Las variables Pregnancies, Glucose, BMI y DiabetesPedigreeFunction mostraron asociaciones positivas y estadísticamente significativas con la presencia de diabetes. Entre ellas, la glucosa presentó la mayor evidencia estadística de asociación. Por otra parte, BloodPressure, SkinThickness, Insulin y Age no resultaron significativas

```
In [81]: #Obtener el accuracy score y la matriz de confusión
from sklearn.metrics import accuracy_score

y_pred = logistic_reg.predict(X_independientes)
accuracy_logistic = accuracy_score(y_dependiente, y_pred)

print("El accuracy score obtenido es de %.8f" %accuracy_logistic)
```

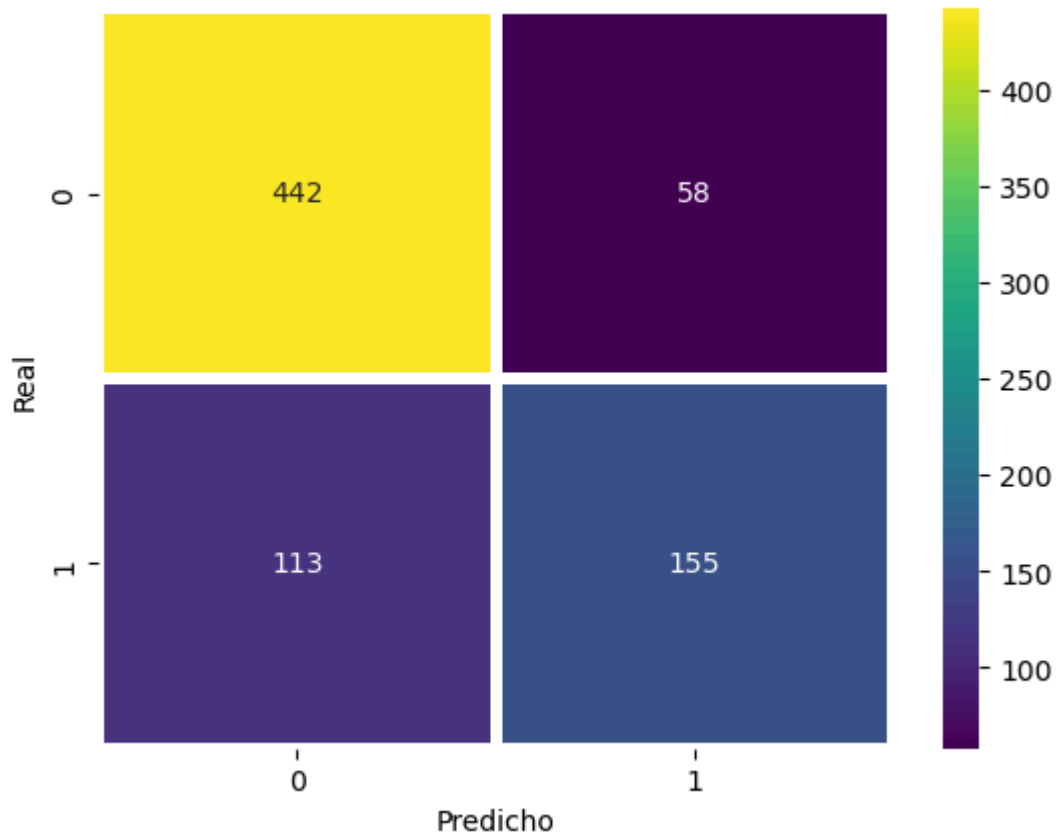
El accuracy score obtenido es de 0.77734375

Esto significa que el modelo de regresión logística clasificó correctamente aproximadamente el 77.73 % de las observaciones evaluadas. En otras palabras, de cada 100 casos, el modelo predijo correctamente alrededor de 78.

```
In [84]: from sklearn.metrics import confusion_matrix

#Obtener la matriz de confusión
cm = confusion_matrix(y_dependiente, y_pred)
sns.heatmap(cm, annot=True, fmt='d', linewidths=3, cmap='viridis')
plt.xlabel("Predicho")
plt.ylabel("Real")
```

Out[84]: Text(50.72222222222214, 0.5, 'Real')



In []: